PROJECT REPORT

Real-Time Room Occupancy Detection System

Using HC-SR501 PIR Sensor & Arduino Uno

Ansh Joseph James

1. Problem Statement

In modern buildings, institutions, and smart environments, energy waste due to unmonitored room occupancy is a growing concern. Lighting systems, air conditioning units, and other electrical appliances are frequently left running in rooms that are unoccupied, leading to unnecessary energy consumption and increased operational costs.

Existing manual systems rely entirely on human discipline to switch off appliances when leaving a room, which is unreliable and inconsistent. While commercial occupancy detection systems exist, they are often expensive, complex to install, and dependent on proprietary software or cloud infrastructure.

Furthermore, conventional PIR-based detection systems suffer from a critical limitation — they detect motion, not presence. This means a person sitting still in a room can be misidentified as absent, causing lights or systems to shut off even when the room is occupied. This creates discomfort and disrupts productivity.

There is therefore a clear need for a low-cost, reliable, real-time room occupancy detection system that can:

- Accurately detect human presence even during periods of minimal movement
- Provide live occupancy status with time-tracking
- Operate independently without internet connectivity or cloud dependency
- Be simple enough for students, hobbyists, and small institutions to build and deploy

2. Scope of Solution

The proposed solution is a Real-Time Room Occupancy Detection System built using an HC-SR501 PIR Sensor and an Arduino Uno microcontroller. The system addresses the problem through the following scope:

2.1 Real-Time Occupancy Detection

- The PIR sensor continuously monitors the room for infrared radiation changes caused by human movement
- The Arduino reads the sensor output and instantly classifies the room as Occupied or Vacant
- An LED indicator provides immediate visual feedback of the current occupancy status

2.2 False Vacancy Prevention (Hold Timer)

- A vacancy hold timer is implemented in software to address the core PIR limitation of not detecting still persons
- The room is only declared vacant after a sustained period of zero motion detections
- Any motion detected during the hold window resets the countdown and prints a re-detection alert

2.3 Occupancy Time Tracking

- The system records and displays how long a room has been occupied and how long it has been vacant
- Durations are formatted in human-readable format (e.g. 2m 35s) and logged to the Serial Monitor in real time
- This data can be used for occupancy pattern analysis and energy usage estimation

2.4 Serial Monitor Logging

- All events — occupancy changes, re-detections, countdowns, and durations — are logged with clear labels
- Provides a lightweight, dependency-free data output readable on any computer without additional software

2.5 Hardware Simplicity & Low Cost

- The entire system uses affordable, widely available components
- Total estimated cost is under Rs.650
- No external power supply, Wi-Fi module, or internet connection is required

2.6 What is Outside the Scope

- The system does not count the number of people in a room, only whether it is occupied or not
- It does not connect to the internet or any cloud service in its current form
- It does not control appliances directly (though relay modules can be added as an extension)
- It is not intended for large-scale commercial deployment without hardware modifications

3. Required Components

3.1 Core Hardware Components

Component	Specification	Qty	Est. Cost
Arduino Uno	ATmega328P, 5V, 16MHz	1	Rs.350
PIR Sensor HC-SR501	5V, 3-pin, 120° detection	1	Rs.80
LED	Any color, 5mm	1	Rs.5
Resistor	220Ω, 14 watt	1	Rs.2
Breadboard	830 tie-point, full size	1	Rs.80
Jumper Wires	Male-to-Male	10-15	Rs.50
USB Cable	Type-B (printer cable)	1	Rs.80

Total Estimated Cost: Rs.650

3. Hardware Specifications

Arduino Uno:

Specification	Value
Microcontroller	ATmega328P
Operating Voltage	5V
Digital IO Pins	14 (6 PWM)
Flash Memory	32KB
Clock Speed	16 MHz
USB Interface	Type-B

HC-SR501 PIR Sensor:

Specification	Value
Operating Voltage	4.5V - 20V
Output Voltage	3.3V (HIGH) / 0V (LOW)
Detection Range	3m - 7m (adjustable)
Detection Angle	120° cone
Hold Time	5s - 300s (adjustable via Tx knob)
Trigger Modes	Single (L) / Repeatable (H)
Warm-up Time	~60 seconds after power on

4. Software & IDE

4.1 Primary Software

Software	Purpose	Version
Arduino IDE	Write, compile and upload code	2.x (latest)
USB Serial Driver (CH340)	Allows PC to detect Arduino via USB	Included with IDE
EasyEDA	Schematic and PCB design	Online (free)

4.2 Arduino IDE Setup

- Go to arduino.cc/en/software and download for your OS (Windows / macOS / Linux)
- Install and launch the IDE
- Go to Tools > Board > Arduino AVR Boards > Arduino Uno
- Go to Tools > Port and select the COM port your Arduino is on
- Paste your code and click the Upload button (arrow icon)

4.3 IDE Settings for This Project

Setting	Value
Board	Arduino Uno
Processor	ATmega328P
Port	COM3 (Windows) / /dev/ttyUSB0 (Linux) / /dev/cu.usbmodem (Mac)
Baud Rate	9600
Programmer	AVRISP mkII (default)

4.4 PC / System Requirements

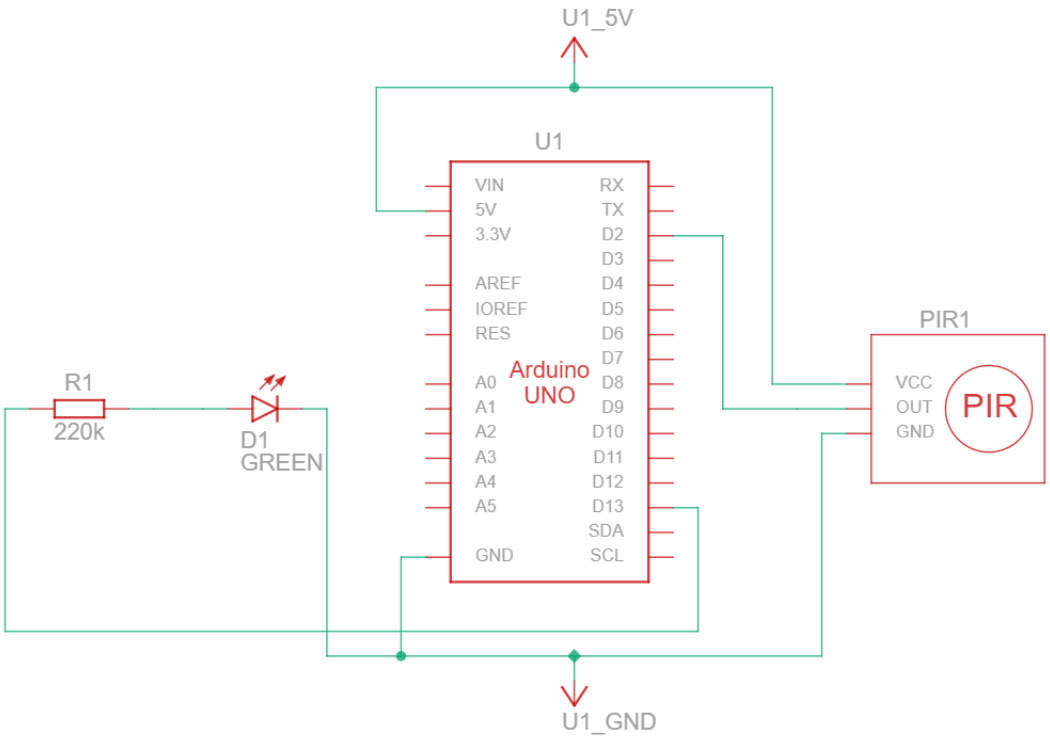
Requirement	Minimum
Operating System	Windows 7+ / macOS 10.10+ / Ubuntu 16.04+
RAM	512MB (2GB recommended)
Storage	500MB free (for Arduino IDE)
USB Port	1 free USB-A port
Internet	Only needed for library downloads

5. Circuit Wiring

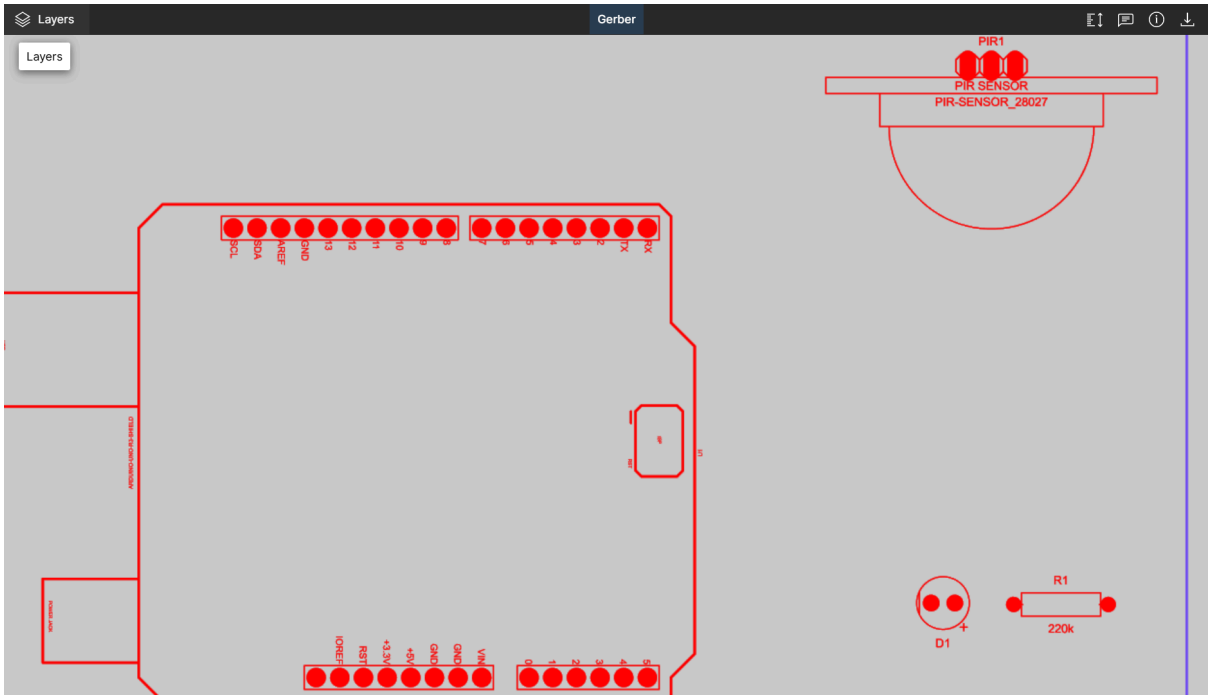
5.1 Connection Table

Component Pin	Connects To
PIR VCC	Arduino 5V
PIR GND	Arduino GND
PIR OUT	Arduino Digital Pin 2
Arduino Pin 13	Resistor (220Ω) then LED Anode (+)
LED Cathode (-)	Arduino GND

5.2 Schematic Diagram



5.3 Gerber Diagram



6. Arduino Code

```
// Code to put into Arduino-Ansh
// Occupancy Detector with Vacancy Hold Timer
// Prevents false "vacant" when person sits still

const int PIR_PIN = 2;//Connection of PIR to pin 2
const int LED_PIN = 13;//Connection of LED to pin 13

bool occupied = false;
unsigned long occupiedSince = 0;
unsigned long lastMotionTime = 0;

// How long with NO motion before declaring vacant (10 seconds)
const unsigned long VACANCY_HOLD_MS = 10000;

String formatDuration(unsigned long ms) {//making the duration shown
in a more readable format
  unsigned long totalSecs = ms / 1000;
  unsigned long mins = totalSecs / 60;
  unsigned long secs = totalSecs % 60;
  if (mins > 0) return String(mins) + "m " + String(secs) + "s";
  return String(secs) + "s";
}

void setup() {
  pinMode(PIR_PIN, INPUT);
  pinMode(LED_PIN, OUTPUT);
  Serial.begin(9600);

  Serial.println("  Room Occupancy Detection System");
  Serial.println("  [Vacancy Hold: 10s]");
  Serial.println("================================");
  Serial.println("System ready! Monitoring...\n");
}

void loop() {
  int motion = digitalRead(PIR_PIN);
  unsigned long now = millis();

  if (motion == HIGH) {
    // Motion detected update last seen time
```

```
    lastMotionTime = now;

    if (!occupied) {
      //Just became OCCUPIED
      occupied = true;
      occupiedSince = now;
      digitalWrite(LED_PIN, HIGH);

      Serial.println(">>> STATUS: OCCUPIED");
      Serial.println("    Motion detected!");
    }
    // If already occupied, just reset the hold timer silently
  }

  if (occupied) {
    unsigned long timeSinceMotion = now - lastMotionTime;
    unsigned long timeOccupied = now - occupiedSince;

    if (timeSinceMotion >= VACANCY_HOLD_MS) {
      // No motion for set time, now truly VACANT
      occupied = false;
      digitalWrite(LED_PIN, LOW);
      Serial.println("    STATUS: VACANT");
                Serial.println("          Was   occupied   for:   "   +
formatDuration(timeOccupied));
      Serial.println("    No motion for 10s");


    } else if (timeSinceMotion > 5000) {
      // Warning that the hold timer is counting down
      unsigned long remaining = (VACANCY_HOLD_MS - timeSinceMotion) /
1000;
            Serial.println("       [Countdown]  Going  vacant  in  "  +
String(remaining) + "s...");
    }
  }

  delay(1000);
}
```

7. Future Extensions

Extension	Purpose	Component Needed
16x2 LCD Display	Show status without a PC	LCD I2C module
Buzzer Alert	Audible notification on change	5V passive buzzer
ESP8266 Wi-Fi	Send data to web dashboard	ESP8266 module
SD Card Logging	Record occupancy history over days	SD card + module
MQTT Integration	Connect to smart home (Home Assistant)	ESP8266 + broker
Relay Module	Auto control lights/appliances	5V relay module
Multiple PIR Sensors	Cover larger rooms or multiple angles	Additional HC-SR501
EasyEDA PCB	Manufacture a custom PCB	EasyEDA + JLCPCB

8. Github Repository Link

https://github.com/Ansh-James/occupancy-detector-project